



Predicción de Insuficiencia Cardíaca mediante Machine Learning

Universidad de las Fuerzas Armadas ESPE

Departamento de Ciencias de la Computación — Inteligencia Artificial

Autores: Miguel Villacres · Miguel Sangucho | **NRC:** 30758

Dataset: `heart.csv` (918 registros · 11 variables predictoras · 1 variable objetivo)

1. Importación de Librerías

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Configuración visual
sns.set_theme(style="whitegrid")
```

2. Carga y Exploración de Datos (EDA)

```
# Cargar el dataset
df = pd.read_csv('2heart.csv')

# Mostrar las primeras filas
display(df.head())

# Información general y revisión de valores nulos
print(df.info())
print("\nValores nulos por columna:\n", df.isnull().sum())

# Distribución de la variable objetivo (ej. 'target' o 'HeartDisease')
# Nota: Cambia 'target' por el nombre real de la columna a predecir en tu CSV
columna_objetivo = 'HeartDisease' # Ajustar si en el CSV se llama diferente (ej. 'HeartDisease')

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x=columna_objetivo, palette='viridis')
plt.title('Distribución de Insuficiencia Cardíaca')
plt.show()
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 918 entries, 0 to 917
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	Age	918 non-null	int64
1	Sex	918 non-null	object
2	ChestPainType	918 non-null	object
3	RestingBP	918 non-null	int64
4	Cholesterol	918 non-null	int64
5	FastingBS	918 non-null	int64
6	RestingECG	918 non-null	object
7	MaxHR	918 non-null	int64
8	ExerciseAngina	918 non-null	object
9	Oldpeak	918 non-null	float64
10	ST_Slope	918 non-null	object
11	HeartDisease	918 non-null	int64

```
dtypes: float64(1), int64(6), object(5)
```

```
memory usage: 86.2+ KB
```

```
None
```

```
Valores nulos por columna:
```

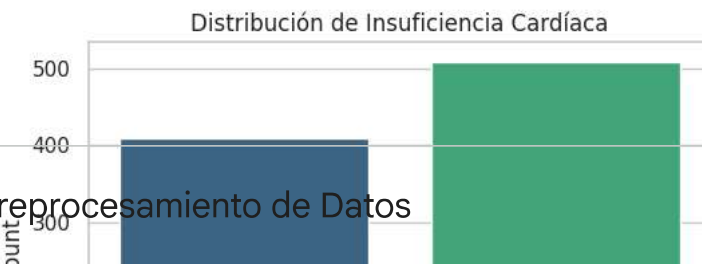
Age	0
Sex	0
ChestPainType	0
RestingBP	0
Cholesterol	0
FastingBS	0
RestingECG	0
MaxHR	0
ExerciseAngina	0
Oldpeak	0
ST_Slope	0
HeartDisease	0

```
dtype: int64
```

```
/tmp/ipykernel_9302/2713404545.py:16: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.countplot(data=df, x=columna_objetivo, palette='viridis')
```



3. Preprocesamiento de Datos

```
# Separar características (X) y la variable objetivo (y)
```

```
X = df.drop(columna_objetivo, axis=1)
```

```
y = df[columna_objetivo]
```

```
# Dividir en datos de entrenamiento y prueba (80% entrenamiento, 20% prueba)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Identificar columnas categóricas
```

```
categorical_cols = X.select_dtypes(include=['object']).columns
```

```
# Aplicar One-Hot Encoding a las columnas categóricas
```

```
X_train_encoded = pd.get_dummies(X_train, columns=categorical_cols, drop_first=True)
```

```
X_test_encoded = pd.get_dummies(X_test, columns=categorical_cols, drop_first=True)
```

```
# Asegurarse de que X_train_encoded y X_test_encoded tengan las mismas columnas
```

```
X_test_encoded = X_test_encoded.reindex(columns=X_train_encoded.columns, fill_value=0)
```

```
# Escalar los datos (importante para variables numéricas con diferentes rangos)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)

# Convertir los arrays escalados de nuevo a DataFrames para mantener las etiquetas de las columnas
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train_encoded.columns, index=X_train_encoded.index)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X_test_encoded.columns, index=X_test_encoded.index)
```

4. Entrenamiento del Modelo

```
# Inicializar y entrenar el modelo
modelo = RandomForestClassifier(n_estimators=100, random_state=42)
modelo.fit(X_train_scaled, y_train)

# Realizar predicciones
y_pred = modelo.predict(X_test_scaled)
```

5. Evaluación y Resultados

```
# Precisión general
precision = accuracy_score(y_test, y_pred)
print(f"Precisión del modelo: {precision * 100:.2f}%\n")

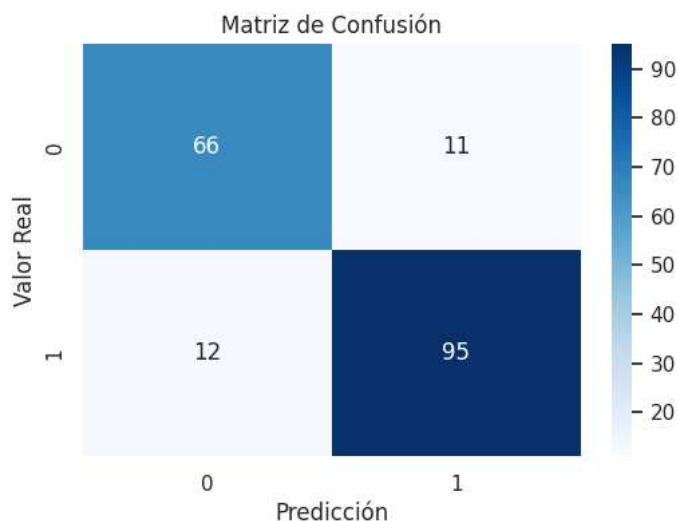
# Reporte de clasificación
print("Reporte de Clasificación:")
print(classification_report(y_test, y_pred))

# Matriz de confusión
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues')
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Valor Real')
plt.show()
```

Precisión del modelo: 87.50%

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.85	0.86	0.85	77
1	0.90	0.89	0.89	107
accuracy			0.88	184
macro avg	0.87	0.87	0.87	184
weighted avg	0.88	0.88	0.88	184



6. Conclusión

```
# Importancia de las características
importancias = modelo.feature_importances_
indices = np.argsort(importancias)[::-1]

plt.figure(figsize=(10, 6))
sns.barplot(x=importancias[indices], y=X_train_encoded.columns[indices], palette='magma')
plt.title('Factores más determinantes para la Insuficiencia Cardíaca')
plt.xlabel('Nivel de Importancia')
plt.ylabel('Característica')
plt.show()
```

/tmp/ipykernel_9302/3191294158.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set

```
sns.barplot(x=importancias[indices], y=X_train_encoded.columns[indices], palette='magma')
```

